SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING PROGRAMME

CSA3717 – SOFTWARE TESTING

EXPERIMENT 9

BANKING APPLICATION – NEGATIVE TEST SCENARIOS AND TEST CASES

Aim:

To Write a case study to identify and analyze the requirements of Banking Applications, and prepare negative test scenarios as well as test cases for the application.

1. Problem Description

A banking application enables authenticated customers to view account details, check balances, transfer funds, pay bills and review transaction history. It must protect customer information and maintain accurate financial records.

The Banking Application is expected to provide reliable, secure and user-friendly services. The testing team verifies the stated requirements using valid and invalid inputs, records actual results, and reports any deviation from the expected behaviour.

2. Functional Requirements

1. Customers must authenticate using valid credentials and any required second factor.
2. Only accounts belonging to the authenticated customer may be displayed.
3. Balance enquiry must show the latest available balance.
4. Fund transfers must validate beneficiary details and account status.
5. Transfer amount must be positive and within transaction limits.
6. Transfers must be rejected when funds are insufficient.
7. Successful transactions must update balances consistently and generate a reference number.
8. Sensitive actions must be protected by authorization and audit logging.

4. Negative Test Scenarios and Test Cases

Negative testing checks whether invalid, incomplete, unauthorized or boundary inputs are safely rejected with a clear message.

Test Case ID	Test Scenario	Test Input / Action	Expected Result	Status
NT01	Login with wrong password	Enter incorrect password	Login is denied.	Pass
NT02	Submit incorrect OTP	Enter invalid or expired OTP	Sensitive action is blocked.	Pass
NT03	Access another customer's account	Attempt to open an account not owned by user	Access is denied.	Pass
NT04	Transfer zero amount	Enter 0 as transfer amount	Transfer is rejected.	Pass
NT05	Transfer negative amount	Enter a negative amount	Transfer is rejected.	Pass
NT06	Insufficient balance	Transfer more than available balance	Transfer is declined without debiting account.	Pass
NT07	Invalid beneficiary	Enter malformed or inactive beneficiary details	Beneficiary validation error is displayed.	Pass
NT08	Duplicate submission	Submit the same transfer request twice rapidly	Only one transaction is posted or duplicate is safely prevented.	Pass

5. Detailed Execution of a Failed Test Case

Test Case ID: NT06

Precondition: The application is running in the test environment and the relevant test data is available.

Test Steps: 1. Sign in if required. 2. Open the relevant feature. 3. Perform the action: Transfer more than available balance. 4. Observe the application response.

Expected Result: Transfer is declined without debiting account.

Actual Result: For demonstration, assume the application accepts the invalid input or does not display the required validation message.

Conclusion: The test is marked as failed for defect-report demonstration. This is an illustrative result, not a claim about a deployed application.

6. Sample Bug Report

Field	Description
Bug ID	BR-009
Linked Test Case	NT06
Severity	High (illustrative)
Priority	High (illustrative)
Bug Title	Banking Application does not correctly validate the condition: Insufficient balance.
Environment	Test environment; browser and operating system should be recorded during actual execution.
Preconditions	The application is running in the test environment and the relevant test data is available.
Steps to Reproduce	1. Sign in if required. 2. Open the relevant feature. 3. Perform the action: Transfer more than available balance. 4. Observe the application response.
Expected Result	Transfer is declined without debiting account.
Actual Result	For demonstration, assume the application accepts the invalid input or does not display the required validation message.
Impact	Invalid data or an unauthorized operation may be accepted, resulting in incorrect processing, data integrity issues or a poor user experience.
Suggested Fix	Apply validation at both client and server sides, return a clear error message, and add regression tests for the boundary and invalid condition.
Status	Open – illustrative sample.

7. Bug Life Cycle

New → Assigned → In Progress → Fixed → Retesting → Closed. If the defect is still reproducible after the correction, the tester reopens it and returns it to the development team.

Conclusion

The test cases prepared for the Banking Application provide coverage of its key functional requirements. Executing them helps identify functional gaps, improve error handling and increase confidence in the quality of the application.